



**Università
di Genova**

High Performance Computing
Report CUDA

Andrea Cattarinich 5137057

November 23, 2025

Contents

1	Architectures	2
2	Hotspot Analysis	3
3	Vectorization	5
4	Parallelization	6
5	Conclusions	7
5.1	Code description	7
5.2	Project Description	7
5.3	Sequential and Parallel Sections	7
5.4	Compilation	7
5.5	Hotspot Identification	7
5.6	Conclusions	7

1 Architectures

For this project, I used two workstations: my personal laptop and the Google Colab environment

HP Envy 17-ae103nl (Laptop)

CPU

- **Model:** Intel Core i7-6500U @ 2.50 GHz
- **Architecture:** x86_64
- **Physical cores:** 2
- **Logical threads:** 4 (Hyper-Threading)

GPU

- **Model:** NVIDIA GeForce GTX 950M
- **CUDA Compute Capability:** 5.0
- **Driver version:** 572.16
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

Google Colab

CPU

- **Model:** Intel(R) Xeon(R) CPU @ 2.20GHz
- **Architecture:** x86_64
- **Physical cores:** 1
- **Logical threads:** 2 (Hyper-Threading)
- **Socket(s):** 1

GPU

- **Model:** NVIDIA Tesla T4
- **CUDA Compute Capability:** 7.5
- **Driver version:** 550.54.15
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

2 Hotspot Analysis

Heat conduction (or heat diffusion) is the process by which thermal energy spreads within a material due to temperature differences. In two dimensions, this phenomenon can be represented as heat spreading across a plate or surface over time. Numerical simulations of heat conduction are widely used in engineering, physics, and high-performance computing to predict temperature distribution and analyze thermal effects.

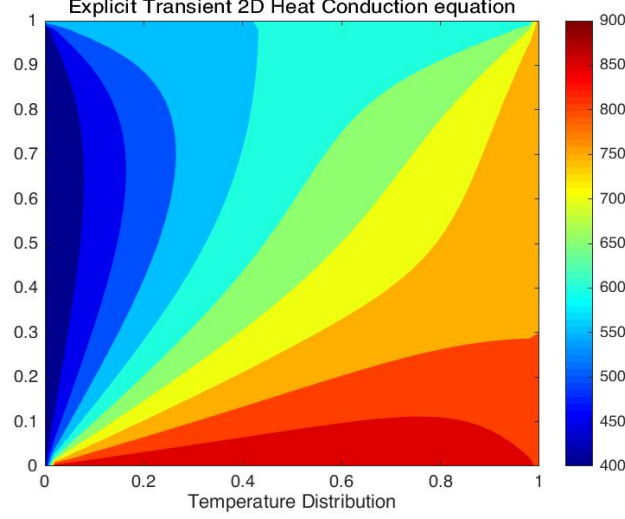


Figure 1: Illustration of 2D heat conduction simulation.

The formula to represent 2D heat conduction is given as follows:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

where:

- T is temperature;
- t is time;
- α is the thermal diffusivity;
- (x, y) are points in a grid.

To solve this problem numerically, one possible finite difference approximation is:

$$\frac{\Delta T}{\Delta t} = \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{(\Delta x)^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{(\Delta y)^2} \right)$$

where:

- Δt is the timestep;
- i, j are indices in the grid;
- n indicates the current timestep, so $T_{i,j}^n$ is the temperature at point (i, j) at time $t = n\Delta t$, and $T_{i,j}^{n+1}$ is the temperature at the next timestep $t = (n+1)\Delta t$.

In the beginning, the boundary points of the grid are assigned with initial temperatures, while the inner points update their temperature iteratively according to the formula above.

TODO: aggiungere una fonte di calore concentrica al centro della griglia.

3 Vectorization

4 Parallelization

5 Conclusions

5.1 Code description

5.2 Project Description

5.3 Sequential and Parallel Sections

5.4 Compilation

5.5 Hotspot Identification

5.6 Conclusions